

Correction-Based Fault Attack Against Randomized MAYO

Mohamed Abdelmonem¹, Lejla Batina², Durba Chatterjee² and Håvard Raddum¹

¹ Simula UiB, Bergen, Norway, {mohamed, haavard}@simula.no

² Radboud University, Nijmegen, The Netherlands, {lejla.batina, durba.chatterjee}@ru.nl

Abstract. This paper introduces a novel fault injection attack targeting the randomized version of the MAYO post-quantum signature scheme. While prior attacks on MAYO either relied on deterministic signing modes or specific memory assumptions, our attack succeeds without such constraints. By exploiting the inherent structural properties of MAYO signatures, we combine targeted fault injections with signature correction techniques to extract partial information about the secret oil space. By systematically accumulating such partial information across multiple fault-induced signatures and utilizing linear dependencies among oil vectors, we present an efficient method for achieving full secret key recovery. The attack requires only one fault injection per oil coefficient, repeated a small (i.e., 8, 17, 10, or 12 for the different MAYO versions, respectively) number of times. We demonstrate the targeted fault injection attack on a MAYO implementation on an ARM Cortex-M4 processor via clock glitching, establishing the feasibility of the attack in practice. Our approach is validated through simulations, and a detailed computational cost analysis is provided. Additionally, we demonstrate the ineffectiveness of some previously proposed countermeasures against our attack, thereby highlighting the urgent need for developing more robust protection mechanisms for multivariate post-quantum signature schemes, such as MAYO.

Keywords: Fault Injection Attack · MAYO · Multivariate cryptography · Post-Quantum Cryptography

1 Introduction

The National Institute of Standards and Technology (NIST) recently finalized its standardization process for Post-Quantum Cryptography (PQC), identifying cryptographic schemes suitable for secure applications against quantum computing threats. Among the selected standards are the digital signature schemes ML-DSA [Nat24a] and SLH-DSA [Nat24b]. To broaden the scope of viable PQC solutions, NIST initiated a second PQC competition in September 2022, focusing on identifying additional signature algorithms offering distinct security assumptions and performance trade-offs relative to ML-DSA and SLH-DSA.

The MAYO signature scheme [Beu22] emerged as a prominent candidate selected by NIST for consideration in the new call for PQC signatures. MAYO belongs to the class of multivariate cryptographic schemes that derive their security from the complexity of solving systems of nonlinear polynomial equations over finite fields. While MAYO offers theoretical assurances of security, practical vulnerabilities associated with physical attacks must also be thoroughly assessed. Several successful side-channel and fault-based attacks on PQC implementations reported in the literature highlight a persistent gap between theoretical resilience and practical robustness. Thus, rigorous evaluation and mitigation of

susceptibility to specific physical attacks should be integral components of PQC deployment preparations.

Recently, significant research attention has been directed towards fault injection attacks against multivariate cryptographic schemes, particularly targeting MAYO [SMA⁺24, AMSU24, JD25, ACK25, BSK25]. However, most existing studies have either relied on theoretical analyses or required specific assumptions for successful demonstrations or focused primarily on vulnerabilities in auxiliary components, such as the hash function used in the MAYO implementation. In this paper, we introduce an attack that exploits a fundamental vulnerability inherent to MAYO's construction, allowing for the recovery of secret keys even in the randomized variant of the algorithm. Furthermore, we demonstrate the ineffectiveness of some of the previously proposed countermeasures, thereby highlighting the urgent need for continued research and the development of more robust and efficient countermeasures.

1.1 Related work

Fault attacks belong to a class of physical attacks that exploit the leakage of sensitive information due to some irregular conditions leading to e.g. code exceptions or faulty computation. With respect to this required faulty behavior, they are considered active, rather than passive side-channel attacks. Namely, side-channel attacks typically just observe signals while the target device is operating under "normal" circumstances. With fault injection (FI) attacks, an attacker attempts to alter environmental or other physical conditions such that the device under attack starts behaving in an unusual way. One way to accomplish this is by "glitching" the device i.e. forcing changes in the values of relevant physical parameters such as clock, voltage or temperature, outside the prescribed intervals. There are numerous non-invasive ways to achieve this goal as follows:

- Clock glitching [ADN⁺10]. In this case a glitch is caused by altering the clock signal. This is typically done with devices that allow the use of an external clock.
- Voltage fault injection (FI) [ABF⁺03]. The attacker can induce this kind of glitch by adding a short positive or negative spike in the power line.
- Electromagnetic fault injection (EMFI) [VR13]. A glitch is caused by emitting a short electromagnetic (EM) pulse towards the device surface causing effects similar to voltage glitching.
- Rowhammer [JLK⁺23]. This is a memory-based attack that abuses the changes in an electric circuit due to the transfer of energy within an electrical network. Those changes can cause faults in dynamic random-access memory (DRAM).

While more expensive methods, such as laser FI, exist, here we focus on "cheap" techniques, such as clock glitching. Also, other non-expensive FI techniques, such as EMFI, could also be used to create similar effects, leading to the secret key recovery. This work is inspired by the idea of *signature correction*, which was first introduced in the context of attacking Dilithium in [IMS⁺22], and later refined in [KPLG24]. We applied the concept to MAYO, and we verified the attack practically on a microcontroller using clock glitching.

The study of fault attacks on multivariate signature schemes began with [HTS11], and was later expanded to target schemes like UOV and Rainbow in [KL19, SK20, AKKM22, FKNT22]. Following the introduction of MAYO [Beu22], many of these existing attacks were shown to be applicable to MAYO as well as demonstrated in [ACK25]. In addition, dedicated fault attacks tailored to MAYO have since been proposed in [JD25, AMSU24, SMA⁺24, BSK25].

In [ACK25], the authors present a comprehensive analysis of all physical attacks on UOV-based signature schemes. They classify each fault attack according to specific attack vectors

and propose practical implementation guidelines alongside dedicated countermeasures to enhance protection against the identified vulnerabilities.

In this work, we demonstrate that signature correction techniques can be effectively combined with fault injection methods to successfully break randomized versions of MAYO, even in the absence of implementation-specific assumptions.

1.2 Contributions

The main contributions of this work are:

1. We show a correction fault attack that targets the component-wise addition of oil and vinegar vectors, and exploits the fact that the final o coefficients of the vinegar vectors are always zero. Since the oil space has dimension o , linear algebra allows efficient reconstruction of the complete oil space, thereby revealing the secret key.
2. We demonstrate the fault injection attack in practice against a randomized MAYO implementation from the pqm4 repository on a CW308-STM32F3 board with an ARM Cortex-M4 STM32F303RBT6 processor. We perform fault injection by clock glitching using Chipwhisperer Husky.
3. We demonstrate that existing countermeasures proposed in prior works are either completely ineffective against our attack or can be circumvented with only minor modifications to the original attack strategy, or come with significantly increased cost.

1.3 Organization of the Paper

The remainder of this paper is structured as follows. In Section 2, we provide a brief introduction to the MAYO signature scheme and offer relevant background regarding known attack strategies. Section 3 presents our proposed fault injection and signature correction attack in detail, including formal descriptions and cost estimation. Section 4 presents the experimental setup details and results of the fault injection attack in practice. Section 5 discusses potential countermeasures against our attack and evaluates their effectiveness. Finally, we summarize our findings and discuss future works in Section 6.

2 Preliminaries

2.1 Notation and Basic Definitions

Let $\mathbb{F}_q$ denote the finite field containing q elements. Vectors of length n over $\mathbb{F}_q$ are denoted as $\mathbb{F}_q^n$ and represented using bold lowercase letters (e.g., $\mathbf{x}, \mathbf{y}$), while matrices over $\mathbb{F}_q$ use bold uppercase letters (e.g., $\mathbf{A}, \mathbf{B}$). The i -th component of vector $\mathbf{x}$ is indicated as $\mathbf{x}[i]$, with indices starting at zero. For indices $0 \leq i < j \leq n$, we write $\mathbf{x}[i : j] \in \mathbb{F}_q^{j-i}$ to represent the subvector comprising the elements $x_i, x_{i+1}, \dots, x_{j-1}$. If $\mathbf{A}$ is a matrix then $\mathbf{A}[i :]$ indicates the i -th row and $\mathbf{A}[:, i]$ the i -th column of $\mathbf{A}$. The concatenation of vectors $\mathbf{u} \in \mathbb{F}_q^a$ and $\mathbf{v} \in \mathbb{F}_q^b$ is denoted by $\mathbf{u} \parallel \mathbf{v} \in \mathbb{F}_q^{a+b}$.

A **multivariate quadratic map** $\mathcal{P}$ over the finite field $\mathbb{F}_q$ consists of m quadratic polynomials in n variables. Each component $p_i(\mathbf{x})$, where $\mathbf{x} = (x_0, \dots, x_{n-1})$, is a multivariate polynomial of total degree at most two with coefficients in $\mathbb{F}_q$. Together, the components form the map $\mathcal{P}$, which defines a function

$$\mathcal{P}(\mathbf{x}) = (p_0, \dots, p_{m-1}) : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m.$$

The **differential** of $\mathcal{P}$ is a bilinear map $\mathcal{P}' : \mathbb{F}_q^n \times \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ and is given by:

$$\mathcal{P}'(\mathbf{x}, \mathbf{y}) = \mathcal{P}(\mathbf{x} + \mathbf{y}) - \mathcal{P}(\mathbf{x}) - \mathcal{P}(\mathbf{y}). \quad (1)$$

When the degree of $\mathcal{P}$ is 2, the differential $\mathcal{P}'$ is a linear mapping when only one of $\mathbf{x}$ or $\mathbf{y}$ are considered as variables.

The **Unbalanced Oil and Vinegar (UOV) trapdoor function** is a multivariate quadratic map $\mathcal{P}$ that vanishes on a secret linear subspace $\mathcal{O} \subset \mathbb{F}_q^n$ of dimension $\dim(\mathcal{O}) = m$, i.e.,

$$\mathcal{P}(\mathbf{o}) = 0 \quad \text{for all } \mathbf{o} \in \mathcal{O}.$$

This linear subspace is called the **oil space** and serves as the trapdoor information. We define $\mathbf{O}$ as a matrix whose columns form a basis of $\mathcal{O}$. We choose the basis matrix, so it is always given in systematic form, meaning it has the structure:

$$\mathbf{O} = \begin{pmatrix} \bar{\mathbf{O}} \\ \mathbf{I}_m \end{pmatrix},$$

where $\mathbf{I}_m$ is the $m \times m$ identity matrix and $\bar{\mathbf{O}} \in \mathbb{F}_q^{(n-m) \times m}$.

Knowing this trapdoor information, it is easy to reconstruct a preimage $\mathbf{x} \in \mathbb{F}_q^n$ of a given salted hash digest $\mathbf{t} \in \mathbb{F}_q^m$ such that $\mathcal{P}(\mathbf{x}) = \mathbf{t}$.

This is done by randomly choosing a vinegar vector $\mathbf{v} \in \mathbb{F}_q^n$ and solving for an oil vector $\mathbf{o} \in \mathcal{O}$ such that $\mathcal{P}(\mathbf{v} + \mathbf{o}) = \mathbf{t}$. Using the differential of $\mathcal{P}$ as defined in (1), we can write

$$\mathbf{t} = \mathcal{P}(\mathbf{v} + \mathbf{o}) = \mathcal{P}'(\mathbf{v}, \mathbf{o}) + \mathcal{P}(\mathbf{o}) + \mathcal{P}(\mathbf{v}). \quad (2)$$

Since $\mathcal{P}(\mathbf{o}) = 0$ by construction and $\mathcal{P}(\mathbf{v})$ is a fixed constant and $\mathcal{P}'(\mathbf{v}, \mathbf{o})$ is linear in $\mathbf{o}$, this yields a system of m linear equations in m variables, which can be solved efficiently. The signature is then given by $\mathbf{s} = \mathbf{v} + \mathbf{o}$. The verifier, using the public map $\mathcal{P}$ and the hash digest $\mathbf{t}$, checks whether $\mathcal{P}(\mathbf{s}) = \mathbf{t}$ holds to validate the signature.

2.2 MAYO Signature Scheme

MAYO is a post-quantum digital signature algorithm based on the UOV framework, designed to offer efficient signatures resistant to both classical and quantum attacks [Beu22]. The primary limitation of traditional UOV signatures is their large key sizes. MAYO addresses this issue by adopting an oil space $\mathcal{O} \subset \mathbb{F}_q^n$ with reduced dimension $o := \dim(\mathcal{O}) < m$, so that the basis matrix becomes

$$\mathbf{O} = \begin{pmatrix} \bar{\mathbf{O}} \\ \mathbf{I}_o \end{pmatrix},$$

with $\bar{\mathbf{O}} \in \mathbb{F}_q^{(n-o) \times o}$. This significantly decreases the key sizes but it also implies that the resulting linear system of equations,

$$\mathcal{P}(\mathbf{v} + \mathbf{o}) = \mathbf{t},$$

often becomes unsolvable, as it comprises m equations in only o unknowns, where $o < m$. To mitigate this issue, MAYO employs a construction that expands the oil space by a factor of k , thus enabling solutions within a higher-dimensional space. This is achieved by defining a new map:

$$\mathcal{P}^*(\mathbf{y}_0, \dots, \mathbf{y}_{k-1}) : \mathbb{F}_q^{kn} \rightarrow \mathbb{F}_q^m,$$

which “whips” the original UOV map $\mathcal{P}$ into a larger one that operates on k vinegar-oil pairs simultaneously. The map is defined as

$$\mathcal{P}^*(\mathbf{y}_0, \dots, \mathbf{y}_{k-1}) := \sum_{i=0}^{k-1} \mathbf{E}_{ii} \mathcal{P}(\mathbf{y}_i) + \sum_{i=0}^{k-1} \sum_{j=i+1}^{k-1} \mathbf{E}_{ij} \mathcal{P}'(\mathbf{y}_i, \mathbf{y}_j),$$

where $\mathbf{E}_{ij} \in \mathbb{F}_q^{m \times m}$ are fixed public matrices and $\mathcal{P}'$ is the differential of $\mathcal{P}$. By choosing parameters such that $ko \geq m$, the new space $\mathcal{O}^k = \{(\mathbf{o}_0, \dots, \mathbf{o}_{k-1}) \mid \mathbf{o}_i \in \mathcal{O}\}$ becomes large enough to ensure that the signer can find a preimage under $\mathcal{P}^*$ with high probability.

The signature consists of k oil vectors masked with vinegar values $\mathbf{v}_0, \mathbf{v}_1, \dots, \mathbf{v}_{k-1} \in \mathbb{F}_q^{n-o}$:

$$\mathbf{s}_i = \mathbf{v}_i \parallel \mathbf{0} + \mathbf{o}_i = (\mathbf{v}_i + \bar{\mathbf{O}}\mathbf{x}_i) \parallel \mathbf{x}_i, \quad (3)$$

where $\mathbf{x}_i \in \mathbb{F}_q^o$, $\mathbf{o}_i \in \mathcal{O}$, and $i \in \{0, \dots, k-1\}$. The parameter sets of MAYO are given in Table 1.

Table 1: Parameter sets of MAYO

Parameter set	MAYO-1	MAYO-2	MAYO-3	MAYO-5
Security level	1	1	3	5
n	86	81	118	154
m	78	64	108	142
o	8	17	10	12
k	10	4	11	12
q	16	16	16	16

MAYO has two versions: deterministic and randomized. In the deterministic version, the vinegar vectors are derived deterministically from the message to be signed, so signing the same message twice will result in the same vinegar vectors. In the randomized version, the vinegar vectors also depend on randomness introduced in the signing process, so signing the same message multiple times will result in different vinegar vectors and also signatures each time. The complete MAYO signing algorithm is given in Algorithm 1.

Algorithm 1 MAYO.Sign(sk, M) [Beu22]

Input: Secret key sk, message M

Output: Signature σ

```

1: (seedsk,  $\mathbf{O}$ )  $\leftarrow$  sk
2: seedpk  $\leftarrow$  SHAKE256(seedsk)
3:  $P^*(\mathbf{x}_0, \dots, \mathbf{x}_{k-1}) \leftarrow \sum_{i=1}^k E_{ii}\mathcal{P}(\mathbf{x}_i) + \left(\sum_{1 \leq i < j \leq k} E_{ij}\mathcal{P}'(\mathbf{x}_i, \mathbf{x}_j)\right)$ 
4:  $R \leftarrow \{0, 1\}^r$   $\triangleright$  Deterministic variant:  $R \leftarrow \{0\}^r$ 
5: salt  $\leftarrow$  SHAKE256( $M \parallel R \parallel \text{seed}_{sk}$ )
6:  $t \leftarrow$  SHAKE256( $M \parallel \text{salt}$ )
7: for ctr from 0 to 255 do
8:    $\mathbf{V} \leftarrow$  SHAKE256( $M \parallel \text{salt} \parallel \text{seed}_{sk} \parallel \text{ctr}$ )
9:    $\mathbf{v}_0, \dots, \mathbf{v}_{k-1} \leftarrow$  Decode( $\mathbf{V}$ )
10:  ( $\mathbf{A}, \mathbf{y}$ )  $\leftarrow$  BuildLinearSystem( $\{\mathbf{v}_0, \dots, \mathbf{v}_{k-1}\}, \mathbf{O}, P^*, \mathbf{t}$ )
11:   $\mathbf{x} \leftarrow$  SampleSolution( $\mathbf{A}, \mathbf{y}$ )  $\triangleright$  Try to find  $\mathbf{Ax} = \mathbf{y}$  (i.e.  $\mathcal{P}^*(s) = t$ )
12:  if  $\mathbf{x} \neq \perp$  then then break
13:  end if
14: end for
15:  $s \leftarrow \{(\mathbf{v}_i + \bar{\mathbf{O}}\mathbf{x}_i) \parallel \mathbf{x}_i\}_{0 \leq i \leq k-1}$ 
16: return  $\sigma = (s, \text{salt})$ 

```

2.3 Key Recovery from a Single Oil Vector

If an attacker knows one oil vector, recovering the entire oil space is merely a matter of solving linear equations by using the reconciliation attack, which was developed against the Rainbow Signature [DYC⁺08]. This attack also works on MAYO [Beu22].

2.4 Signature Correction Attacks

Signature correction attacks have been applied to the Dilithium scheme [IMS⁺22, KPLG24]. The core idea behind these attacks is to induce a fault during the signing process, resulting in a faulty signature where a portion of the secret is omitted. The attacker then attempts to correct the faulty signature by exhaustively guessing the missing part and checking each candidate using the public verification algorithm. Since the verification procedure reveals whether the correction was successful, it effectively serves as an oracle, allowing the attacker to recover the missing secret information.

2.5 Existing Fault attacks against MAYO

Fault injection attacks on the MAYO signature scheme were previously discussed in [JD25, AMSU24, SMA⁺24, BSK25]. In [SMA⁺24], the authors propose two different scenarios for faulting the signing algorithm, which both lead to key recovery. In the first scenario, they assume that vinegar vectors are initialized to a constant known value before sampling, which, for simplicity, they assume to be zero. A loop-abort fault in this case results in one or more vinegar vectors retaining this initial constant. If the sampling of the $(k-1)$ -th vinegar vector is skipped, the resulting faulted signature is

$$\mathbf{S}' = (\mathbf{s}_0, \dots, \mathbf{s}'_{k-1}) = ((\mathbf{v}_0 || \mathbf{0}) + \mathbf{o}_0, \dots, \mathbf{0} + \mathbf{o}_{k-1}).$$

Since $\mathbf{s}'_{k-1} = \mathbf{o}_{k-1}$, we have therefore found one oil vector and the whole oil space can be recovered using the reconciliation attack [DYC⁺08].

The second scenario utilizes a differential fault attack and only works in the deterministic setting. The attacker first obtains a correctly computed signature $\mathbf{S} = (\mathbf{s}_0, \dots, \mathbf{s}_{k-1})$, with

$$\mathbf{s}_{k-1} = (\mathbf{v}_{k-1} || \mathbf{0}) + \mathbf{o}_{k-1},$$

then injects a loop-abort fault during the computation of a second signature $\mathbf{S}'$ where

$$\mathbf{s}'_{k-1} = (\mathbf{v}_{k-1} || \mathbf{0}) + \mathbf{0}.$$

Taking the difference reveals one of the oil vectors

$$\mathbf{s}_{k-1} - \mathbf{s}'_{k-1} = \mathbf{o}_{k-1} \in \mathcal{O}.$$

This attack only works in the deterministic version, where the vinegar only depends on the message. In the randomized version, we have different vinegar vectors even for the same message, so this attack will not work.

In [AMSU24], the authors demonstrate that this attack may also succeed in the randomized mode, provided at least one vinegar value in the faulted signature matches a vinegar value in the correct signature. This scenario can occur if the memory holding the vinegar vectors is allocated on the stack without being properly initialized prior to sampling. Consequently, vinegar vectors might retain values from the previous signature generation process.

In [ACK25], the authors propose disturbing the setup of the linear system in line 10 of Algorithm 1, which leads to an alternative solution $\mathbf{x}'_{k-1}$. Taking the difference between a correct signature and a faulty signature yields

$$\begin{aligned} \mathbf{s}'_{k-1} - \mathbf{s}_{k-1} &= (\mathbf{v}_{k-1} + \bar{\mathbf{O}}\mathbf{x}'_{k-1}) || \mathbf{x}'_{k-1} - (\mathbf{v}_{k-1} + \bar{\mathbf{O}}\mathbf{x}_{k-1}) || \mathbf{x}_{k-1} \\ &= \bar{\mathbf{O}}(\mathbf{x}'_{k-1} - \mathbf{x}_{k-1}) || (\mathbf{x}'_{k-1} - \mathbf{x}_{k-1}) \in \mathcal{O}. \end{aligned}$$

Since this approach relies on the vinegar vector $\mathbf{v}_{k-1}$ being identical in both signatures, the attack requires a deterministic signing setting.

In [BSK25], the authors present an alternative method for obtaining a different solution $\mathbf{x}'_{k-1}$, which can be exploited in the same manner. They fault the target vector t in line 6 of Algorithm 1 by attacking the SHAKE algorithm, again resulting in a distinct solution $\mathbf{x}'_{k-1}$. The attack is demonstrated using a clock-glitching fault injection, but it is likewise limited to the deterministic setting.

A similar attack on the SHAKE algorithm was introduced in [JD25]. But here, the authors attack the SHAKE algorithm, which computes the seed used for sampling the vinegar values in line 8 in Algorithm 1 instead. As a result, the faulty vinegar vector becomes a constant known to the attacker. Both approaches assume that the hash function used to compute the target vector t or the vinegar seeds can be faulted. However, countermeasures against such attacks have been proposed, for instance by protecting the hashing process with a fault-tolerant Keccak implementation, as described in [GOW⁺24, MBM20].

In this work, we present a fault injection attack on randomized MAYO, which does not require any memory assumptions and does not necessitate the ability to fault the hashing algorithm used. Our attack assumes the vinegar vectors are unknown quantities, freshly and independently generated for each call to the signature algorithm. We demonstrate that this attack can be done using clock glitching and we show that existing countermeasures are either insufficient to prevent this fault attack, or can be easily circumvented with slight adjustments, or are expensive to implement. In Table 2, we compare existing attacks with our newly proposed attack.

Table 2: Comparison of fault attacks on the MAYO signature scheme.

Source	Attack tion	Descrip- tion	Assumption	Evaluation
[SMA ⁺ 24]	Skip vinegar to oil ad- dition	None		Theoretical
	Skip oil to vinegar ad- dition	Deterministic		Theoretical
[AMSU24]	Fix vinegar vector	Zero-initialization/ exact memory reuse		Practical
[ACK25]	Disturb linear sys- tem setup	Deterministic		Theoretical
[BSK25]	Fault target vector/ solution sampling	Deterministic, Unprotected hashing		Practical
[JD25]	Fix vinegar seeds	Unprotected hashing		Practical
This work	skip oil to vinegar addition component- wise	None		Practical

3 Proposed Fault Injection Attack against MAYO

Our proposed attack consists of two main steps:

1. Fault Injection and Signature Correction:
The goal of this step is to extract partial information about secret oil vectors by inducing faults during signature generation and exploiting the verification algorithm as an oracle.
2. Recovery of Oil Space:
In this step, we reconstruct the basis of the oil space using the partial information gathered in Step 1 and linear algebra.

We describe the two steps in detail in the following subsections. The steps can be done either in a fully serialized way, so Step 1 is fully completed before Step 2 is started, or in an interleaved way where parts of the basis matrix $\bar{\mathbf{O}}$ are recovered as soon we have enough information to do so. The explanation below follows the second approach, recovering one row of $\bar{\mathbf{O}}$ for every o faulted signatures.

3.1 Fault Injection and Signature Correction

Our fault attack targets the addition of one oil vector to the vinegar vector in line 15 of Algorithm 1. It suffices to successfully target just one of the signature components in

$$\mathbf{S} = (\mathbf{s}_0, \dots, \mathbf{s}_{k-1}),$$

for example

$$\mathbf{s}_0 = (\mathbf{v}_0 + \bar{\mathbf{O}}\mathbf{x}_0) \parallel \mathbf{x}_0.$$

As we only need to focus on a single signature vector and the specific index is irrelevant, we will omit the subscript from this point onward and simply write $\mathbf{s}, \mathbf{v}, \mathbf{o}$ instead of $\mathbf{s}_0, \mathbf{v}_0, \mathbf{o}_0$:

$$\mathbf{s} = (\mathbf{v} + \bar{\mathbf{O}}\mathbf{x}) \parallel \mathbf{x}. \quad (4)$$

What has been done so far in [SMA⁺24, AMSU24] is to induce a fault that skips the addition of the oil vector entirely. This results in a faulty signature of the form

$$\mathbf{s}' = \mathbf{v} \parallel \mathbf{0},$$

where the oil part is zeroed out. Instead of signing again and applying a differential attack, one can attempt to recover the correct signature by correcting the faulty one. This is done by testing all possible oil candidates $\alpha \in \mathbb{F}_q^o$ and computing

$$\mathbf{s}'' = \mathbf{v} \parallel \mathbf{0} + \alpha.$$

For each candidate, we use the verification algorithm to check whether the modified signature verifies correctly. If verification succeeds for some α , we conclude that α is a valid oil vector. Since a single such vector is sufficient to recover the entire oil space according to [DYC⁺08], this would break the scheme. However, this naive approach requires testing up to q^n possible α 's, making the attack computationally infeasible for the parameter sizes used in MAYO.

A natural refinement is to inject a fault that affects only a single coefficient of the signature vector $\mathbf{s}$, and attempt to correct it by guessing a single value $\alpha_i \in \mathbb{F}_q$ instead of an entire vector. Specifically, we inject the fault and correct it in only one component, to get

$$\mathbf{s}''[i] = \mathbf{v}[i] + \alpha_i$$

for some index $i \in \{0, \dots, n - o - 1\}$. This reduces the complexity of the attack to at most q verification queries. However, it does not recover the full oil vector; instead, it only reveals the value of a single coefficient of some vector in the oil space:

$$\mathbf{o}[i] = \alpha_i,$$

whereas the remaining components of the oil vector remain unknown.

At first glance, it may seem that repeating this process for all indices i would allow recovery of the entire oil vector. However, each signature uses a freshly sampled oil vector drawn from the same oil subspace. As a result, each recovered coordinate α_i originates from a different oil vector. Concatenating these values does not necessarily yield a vector within the oil space, and therefore does not lead to recovery of the secret. Instead, we follow the three steps outlined below and explain in the next section how this approach can be used to recover the entire oil space.

1. **Inject faults to create defective signatures.**

For each $i \in \{0, \dots, n - o - 1\}$, perform a fault injection which skips the addition of the oil vector's coefficient i . This produces faulty signatures $\mathbf{s}'$ where

$$\mathbf{s}'[j] = \begin{cases} \mathbf{v}[j] & \text{for } j = i \\ \mathbf{v}[j] + \mathbf{o}[j] & \text{for } j \neq i. \end{cases}$$

2. **Recover oil components using the verifier as an oracle.**

For each faulty signature $\mathbf{s}'$, test values $\alpha \in \mathbb{F}_q$ such that $\mathbf{s}''$ where

$$\mathbf{s}''[j] = \begin{cases} \mathbf{s}'[j] + \alpha & \text{for } j = i \\ \mathbf{s}'[j] & \text{for } j \neq i \end{cases}$$

verifies successfully. When this happens, we conclude

$$\alpha = \mathbf{o}[i]$$

for the particular oil vector $\mathbf{o}$ chosen during signing.

3. **Accumulate partial oil vectors.**

Repeat this process o times for each index i to end up with $o(n - o)$ partial oil vectors, each revealing the i -th component. For each of these partial oil vectors, we also save the last o components of the signature, namely the $\mathbf{x} \in \mathbb{F}_q^o$ used to produce the particular oil vector chosen in each signature.

3.2 Recovery of Oil Space

When generating a faulty signature targeting index i and using the correction oracle, we obtain an oil vector where the coefficients with indices in $\{i, n - o, n - o + 1, \dots, n - 1\}$ are known. After finding o such oil vectors, we put them as column vectors in an $n \times o$ matrix $\mathbf{A}_i$:

$$\mathbf{A}_i = \begin{bmatrix} * & \dots & * \\ \vdots & & \vdots \\ a_{i,0} & \dots & a_{i,o-1} \\ * & \dots & * \\ \vdots & & \vdots \\ a_{n-o,0} & \dots & a_{n-o,o-1} \\ \vdots & & \vdots \\ a_{n-1,0} & \dots & a_{n-1,o-1} \end{bmatrix}, \quad (5)$$

where $*$ indicates unknown values.

Denote by $\mathbf{X}_i$ the $o \times o$ submatrix of $\mathbf{A}_i$ consisting of the last o rows. When building the matrix $\mathbf{A}_i$, we ensure that $\mathbf{X}_i$ has full rank. This is done by checking if the $\mathbf{X}_i$ -part of a new column to be added to $\mathbf{A}_i$ is linearly dependent on existing columns in $\mathbf{X}_i$. If it is, we reject the column and sample another one. After the complete $\mathbf{A}_i$ is constructed, we know that $\mathbf{X}_i$ is an invertible matrix.

Using $\mathbf{X}_i^{-1}$ to change the basis of the columns in $\mathbf{A}_i$, we get

$$\mathbf{A}_i \mathbf{X}_i^{-1} = \begin{bmatrix} * \\ \mathbf{A}_i[i :] \mathbf{X}_i^{-1} \\ * \\ \mathbf{I}_o \end{bmatrix}.$$

Algorithm 2 Recovering oil space

Require: Ability to generate faulty MAYO signatures where the addition of any oil vector coefficient is skipped

Ensure: Basis of oil space $\mathcal{O}$

```

1:  $M \leftarrow$  random message
2:  $\bar{\mathbf{O}} \leftarrow \mathbb{I}^{n-o \times o}$  ▷ Empty matrix of size  $(n - o) \times 0$ 
3: for  $0 \leq i < n - o$  do
4:    $\mathbf{a} \leftarrow \mathbb{I}^o$  ▷ Empty vector of length  $o$ 
5:    $\mathbf{X} \leftarrow \mathbb{I}^{o \times o}$  ▷ Empty matrix of size  $o \times o$ 
6:    $t \leftarrow 0$ 
7:   while  $t < o$  do
8:      $\mathbf{s} \leftarrow$  signature on  $M$  where addition of  $\mathbf{o}[i]$  is skipped
9:      $\mathbf{x} \leftarrow \mathbf{s}[n - o : n]$ 
10:    if  $\mathbf{x}$  is linearly independent of existing columns in  $\mathbf{X}$  then
11:       $\mathbf{s}' \leftarrow \mathbf{s}$ 
12:      for  $\alpha \in \mathbb{F}_q$  do
13:         $\mathbf{s}'[i] \leftarrow \mathbf{s}[i] + \alpha$ 
14:        if  $\text{verify}(M, \mathbf{s}')$  returns true then
15:          break
16:        end if
17:      end for
18:       $\mathbf{X}[:, t] \leftarrow \mathbf{x}$ 
19:       $\mathbf{a}[t] \leftarrow \alpha$ 
20:       $t \leftarrow t + 1$ 
21:    end if
22:  end while
23:   $\mathbf{X}^{-1} \leftarrow$  inverse of  $\mathbf{X}$ 
24:   $\bar{\mathbf{O}}[i :] \leftarrow \mathbf{a}\mathbf{X}^{-1}$ 
25: end for
26: return  $\bar{\mathbf{O}}$ 

```

The columns in $\mathbf{A}_i \mathbf{X}_i^{-1}$ are therefore the same as in $\mathbf{O}$, so multiplying the indices in row i of $\mathbf{A}_i$ with $\mathbf{X}_i^{-1}$ recovers row i of the secret oil space: $\bar{\mathbf{O}}[i :] = \mathbf{A}_i[i :] \mathbf{X}_i^{-1}$.

Repeating this process for $0 \leq i < n - o$ will recover the basis for the entire secret oil space. The complete algorithm for recovering the oil space from corrected faulty signatures is given in Algorithm 2. We show the attack procedure on a small example.

Example 3.1. $n = 6, o = 2, q = 17$

Let the secret oil space $\mathcal{O} \subset \mathbb{F}_q^n$ be defined by the basis Matrix

$$\mathbf{O} = \begin{pmatrix} 10 & 7 \\ 11 & 3 \\ 2 & 2 \\ 3 & 0 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Using the verification oracle, assume we have recovered the following two partial vectors in the oil space.

$$\mathbf{o}_0 = \begin{pmatrix} 16 \\ * \\ * \\ * \\ 6 \\ 1 \end{pmatrix}, \quad \mathbf{o}_1 = \begin{pmatrix} 2 \\ * \\ * \\ * \\ 13 \\ 6 \end{pmatrix}.$$

This gives

$$\mathbf{X}_0 = \begin{pmatrix} 6 & 13 \\ 1 & 6 \end{pmatrix}, \quad \mathbf{a} = (16 \quad 2),$$

where $\mathbf{a}$ is the first row of $\mathbf{A}_0$. Inverting $\mathbf{X}_0$ yields

$$\mathbf{X}_0^{-1} = \begin{pmatrix} 1 & 12 \\ 14 & 1 \end{pmatrix}.$$

The multiplication $\mathbf{a}\mathbf{X}_0^{-1}$ then gives

$$(16 \quad 2) \begin{pmatrix} 1 & 12 \\ 14 & 1 \end{pmatrix} = (10 \quad 7),$$

which is equal to $\bar{\mathbf{O}}[0 :]$.

3.3 Cost analysis

In this section, we analyze the theoretical costs associated with our attack, assuming skipping the addition is always successful. The attack is divided into two main parts: the accumulation of partial information about the oil space and the recovery of the secret key. After successfully collecting the necessary oil vector information, the secret key can be efficiently recovered. This part consists of:

- inverting $n - o$ matrices, each of size $o \times o$.
- Executing $n - o$ vector/matrix multiplications, each involving vectors of length o and matrices of size $o \times o$.

For the largest MAYO variant, this translates into doing $2^{17.9}$ field multiplications for inverting the matrices. These are small complexities, and using simulated faults, we were able to recover the whole space within seconds for all MAYO parameter sets.

To accumulate the necessary partial information from the oil space, the attack theoretically requires:

- $(n - o)o$ signatures generated with targeted fault injections. In practice, this number is slightly higher, as we may need to sample more than o faulty signatures before the matrices $\mathbf{X}_i$ have full rank. For $q = 16$ and $o = 8$ the expected number of vectors to sample to have a full-rank matrix $\mathbf{X}_i$ is only 1.07 o [DF04, p. 35]. We have also verified experimentally that the $\mathbf{X}_i$ matrices we get in practice are almost always invertible. We therefore ignore this slight increase in the number of faulty signatures needed from here on.
- $q(n - o)o$ verification attempts in the worst-case scenario, to successfully correct the oil vector coefficients from the faulty signatures.

For the largest MAYO variant, this translates to running the verification oracle at most $2^{14.7}$ times, which is feasible.

Assuming we need only $q/2 = 8$ verifications, the total expected computational cost for the fault injection and correction phase can be calculated as follows:

$$\text{Total Time} = N_{\text{sig}} \cdot t_{\text{sign}} + 8 \cdot N_{\text{sig}} \cdot t_{\text{verify}}, \quad (6)$$

where N_{sig} represents the number of required signatures, and t_{sign} and t_{verify} denote the average signing and verification times, respectively.

Faulting multiple coefficients Injecting faults into exactly one coefficient can be challenging in practical settings, depending on the fault injection method, and can even be impossible if the assembly code of an implementation adds 4 or 8 oil coefficients in the same instruction. Note that our attack can still handle such cases, albeit at the expense of having to guess the values of multiple coefficients simultaneously. If the addition of l oil coefficients is skipped, the corrected signature can be recovered by exhaustively testing all possible corrections in $\mathbb{F}_q^l$ until verification succeeds. While this strategy reduces the total number of required faulty signatures by a factor l , it exponentially increases the number of verification attempts per signature to q^l verifications for each faulty signature. In this case, the average cost can be computed as follows:

$$\text{Total Time} = N_{\text{sig}}/l \cdot t_{\text{sign}} + 16^l/2 \cdot N_{\text{sig}} \cdot t_{\text{verify}}.$$

If l is 4 or 8, the complexity for accumulating the oil coefficients increases to $2^{24.7}$ or $2^{39.7}$ in the worst case, respectively. These complexities are still practical.

Transferability to UOV The same attack can be applied to the original UOV algorithm submitted to the NIST competition (see line 11 of the signing algorithm in [BCD⁺23]); however, it becomes more expensive in this case because the field size q is larger for most parameter sets, as shown in Table 3. In Table 4, we give an overview of how many signatures and expected verifications are needed for MAYO and UOV to perform this attack over all security parameters, assuming we skip the addition of $l = 1, l = 4$, or $l = 8$ indices at a time.

Table 3: Parameter sets of UOV

Parameter set	uov-Ip	uov-Is	uov-III	uov-V
Security level	1	1	3	5
n	112	160	184	244
m	44	64	72	96
q	256	16	256	256

4 Experimental Results

In this section, we first introduce our experimental setup, followed the attacker model and fault injection results on a reference implementation of **MAYO-1**. We demonstrate that we are able to skip the addition of single oil coefficients of our choosing, such that only the vinegar value remains in the particular chosen location of the signature vector.

4.1 Experimental Setup

The MAYO implementation is evaluated on a CW308-STM32F303 target mounted on a CW308 UFO board. The target board houses an ARM Cortex-M4 STM32F303RBT6

Table 4: Number of signatures and verifications for MAYO and UOV across parameter sets and $\ell \in \{1, 4, 8\}$.

Algorithm	Signatures			Verifications		
	$\ell = 1$	$\ell = 4$	$\ell = 8$	$\ell = 1$	$\ell = 4$	$\ell = 8$
MAYO-1	624	156	78	$\approx 2^{12}$	$\approx 2^{24}$	$\approx 2^{40}$
MAYO-2	1088	272	136	$\approx 2^{13}$	$\approx 2^{25}$	$\approx 2^{41}$
MAYO-3	1080	270	135	$\approx 2^{13}$	$\approx 2^{25}$	$\approx 2^{41}$
MAYO-5	1704	426	213	$\approx 2^{14}$	$\approx 2^{26}$	$\approx 2^{42}$
uov-1p	2992	748	374	$\approx 2^{19}$	$\approx 2^{41}$	$\approx 2^{72}$
uov-Is	6144	1536	768	$\approx 2^{16}$	$\approx 2^{26}$	$\approx 2^{41}$
uov-III	8064	2016	1008	$\approx 2^{20}$	$\approx 2^{42}$	$\approx 2^{73}$
uov-V	14208	3552	1776	$\approx 2^{21}$	$\approx 2^{43}$	$\approx 2^{74}$

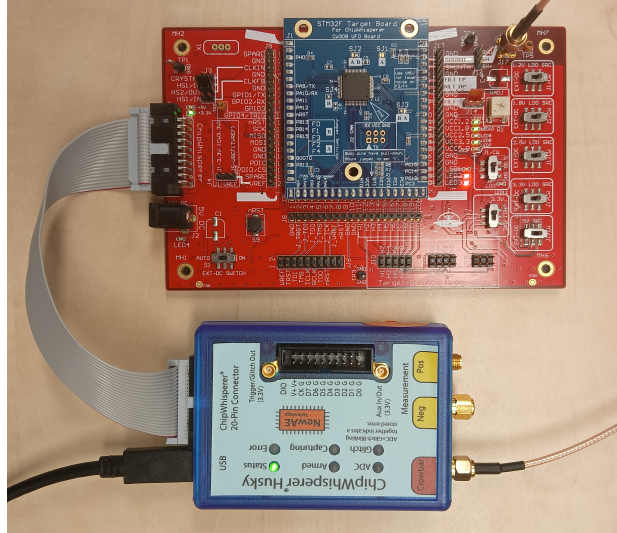


Figure 1: Experimental Setup for fault injection. The STM32F3 target platform (top) executes the MAYO implementation, while the ChipWhisperer Husky (bottom) introduces controlled glitches into the clock signal to induce faults during execution.

processor, operating at 7.37MHz. We use the **MAYO-1** reference implementation from the pqm4 repository ¹, a benchmarking framework for post-quantum cryptographic implementations for the ARM Cortex-M4 microcontroller. The C code is compiled using the `arm-none-eabi-gcc` compiler (version 13.2) and the default optimization level `-O0`. Faults are injected using the CW-Husky platform via clock glitching. Figure 1 shows the experimental setup, comprising the target board (top) and the CW-Husky (bottom). Short glitches are injected into the clock signal via the SMA connector from the Crowbar port on the Husky, potentially resulting in instruction skip or change of instruction operands. The 20-pin connector facilitates serial communication and handles trigger and reset signals.

¹<https://github.com/PQCMayo/pqm4/>

Listing 1 C code snippet of the MAYO signature generation, focusing on the addition of oil and vinegar vectors.

```

1  for (int i = 0; i <= k - 1; ++i) {
2      vi = Vdec + i * (n - o);
3      mat_mul(sk.0, x + i * o, 0x, o, n - o, 1);
4      trigger_high();
5      mat_add(vi, 0x, s + i * n, n - o, 1);
6      trigger_low();
7      memcpy(s + i * n + (n - o), x + i * o, o);
8  }
9  encode(s, sig, n * k);

```

4.2 Fault Injection Attack on MAYO

Attacker model. We consider an attacker with physical access to the board under attack. The attacker needs the same type of board to experimentally find the glitching parameter sets that skips the addition of one, or a few, oil coefficient(s) in the chosen coordinates of a signature vector. The adversary is assumed to have the capability to trigger executions of the signing procedure on the target device and to inject precisely timed glitches in clock signal.

Target Implementation Details. The signing algorithm takes 2.3 seconds on the target board. This corresponds to a reference implementation compiled with optimization level -O0. We target the first iteration of the matrix addition operation (line 15 of Algorithm 1). Listing 1 lists the C code snippet corresponding to this operation, which we target in the fault injection attack. We focus on the first invocation to the `mat_add` operation in the loop within the MAYO signature function. The `mat_add` function call is responsible for adding the oil vector (represented by `0x`) to the vinegar vector (represented by `vi`). It comprises a loop performing an iterative addition of the coefficients. For targeted injection, we modify the C implementation to set the trigger signal only during the execution of the first call to the `mat_add` function.

Listing 2 depicts the corresponding assembly instructions executed in the first iteration. As shown in lines 2 and 3, the implementation loads the current vinegar and oil coefficients into registers `r5` and `r2`, respectively. Line 4 updates the loop iterator. The arithmetic combination of the two operands occurs at line 5, where the addition $r5 \leftarrow r5 + r2$ is performed and the result is written back to `r5`. Thus skipping this instruction suppresses the intended incorporation of the oil component and causes the original vinegar value being retained in `r5`. This is the behavior we intend to achieve for the fault attack. To achieve this expected behavior, we first scan the fault parameters, details of which we now describe.

FI parameters and outcomes. In the presence of fault injection, each execution can result in one of the three following scenarios:

1. *normal execution* where the fault does not impact the execution;
2. *faulty execution* wherein the injected fault perturbs the execution and produces an incorrect signature;
3. *reset*, which initializes a system reset to stabilize the target.

These scenarios are easily distinguished by observing the signature.

Listing 2 Assembly code snippet of the MAYO signature generation, focusing on the addition of oil and vinegar vectors.

```

1 LAB_08000636:
2     ldrb.w r5, [r0], #1 ; r5 = *Vdec; Vdec++ (Vinegar)
3     ldrb.w r2, [r3], #1 ; r2 = *0x; 0x++ (oil)
4     adds r4, #1 ; i++ (loop iterator)
5     eors r5, r2 ; r5 = r5 ^ r2 (Addition of vinegar and oil)
6     cmp r4, #78
7     strb.w r5, [r1], #1 ; (storing vinegar + oil)
8     bne LAB_08000636

```

The fault characterization was performed over a broad range of glitch width, offset, and external offset parameters, and for different messages chosen at random. The width controls the duration of the glitch, offset controls the position/timing of the glitch with respect to the positive edge of the clock signal, and external offset decides the number of clock cycles that are passed before the glitch is introduced. Since the target instruction is executed in a loop where the coefficients are computed sequentially, the external offset determines which coefficient(s) get impacted. Based on the results from the parameter scan, we narrowed down the width and offset parameter range.

Identifying glitch parameters for a single fixed input. For each position in the first signature vector, we perform a search for glitch parameters that successfully skip the addition of a given oil coefficient. Glitching the addition operation can result in three possibilities: the resultant register holds the vinegar value, register holds an oil value or the register contains a random value. Among these three cases, for the proposed correction attack, we are interested in the scenario of the vinegar value residing in the destination register. We performed a wide scan for width offset and external values. Figure 2 depicts the distribution of fault parameters for different coefficients corresponding to executions resulting in vinegar values in the destination register. This demonstrates that we can effectively skip the addition of the oil coefficient in all indices, as depicted in the figure.

From this figure, we can observe that the external offset varies by an interval of approximately 10 clock cycles for consecutive coefficient indices. We obtain these results for glitch width in the range [25-40] and for offset in the range [10-24], respectively, implying that narrow glitches are more potent in skipping the addition. A glitch offset value less than 24 introduces a falling edge immediately after the clock rising edge. This shortens the high phase of the clock signal, causing the instruction triggered at the rising edge to be skipped. Both glitch width and offset are measured in terms of clock phase shifts. Note that these parameters were obtained from a wide, albeit non-exhaustive scan, and hence are not necessarily optimal.

Fault Injection success rate for varying inputs. This corresponds to the realistic scenario, where every time, a different input is fed to the signature algorithm. To determine the success probability of the fault injection, we tested each parameter in the identified parameter set 20 times with different inputs (since we are targeting a randomized scheme) to compute how consistently it skipped the addition of the oil coefficient. Figure 3 depicts the success rate for each index computed over 20 inputs (different than the ones used to identify the glitch parameters). We can observe that for most indices, the success rate is greater than 0.7, while for every fourth index, the success rate dips to 0.35-0.5. This behaviour is consistent across different inputs, indicating this is independent of the input data. To the best of our understanding, this phenomenon can be attributed to the microarchitectural effects.

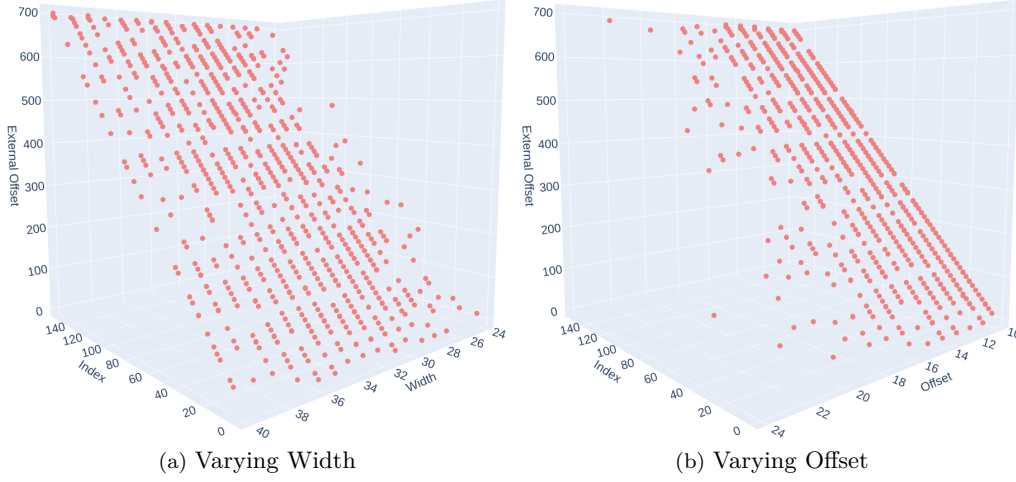


Figure 2: Parameters (Width, Offset, External Offset) resulting in successful fault injections that bypass the addition operation at different coefficient indices (denoted as Index).

4.3 Practical evaluation

Let F_i be the fault parameters set (width, offset, external offset) that target only index i . A successful fault at index i means we have skipped the addition of the oil to the vinegar, so the signature at index i only contains the vinegar value, but is correct everywhere else. The oil value for this index can then be recovered easily using the verification oracle. If we do this o times, then we will get $\mathbf{A}_i[i :]$.

Our experiments have shown that faults are not always successful, meaning that some of the entries of $\mathbf{A}_i[i :]$ may be incorrect. We now explain how the basic attack, assuming always successful faults, can be modified to account for some of the faults being unsuccessful.

Let $p_i = \Pr(F_i \text{ is successful})$ be the probability that the parameters F_i successfully skip the addition in the i -th index. To estimate p_i and determine the optimal F_i to use in our attack on that board, we ran the signing algorithm 20 times and measured the success rates for various fault parameters. In Figure 3, we plot the success rate of the most successful fault parameter for each index.

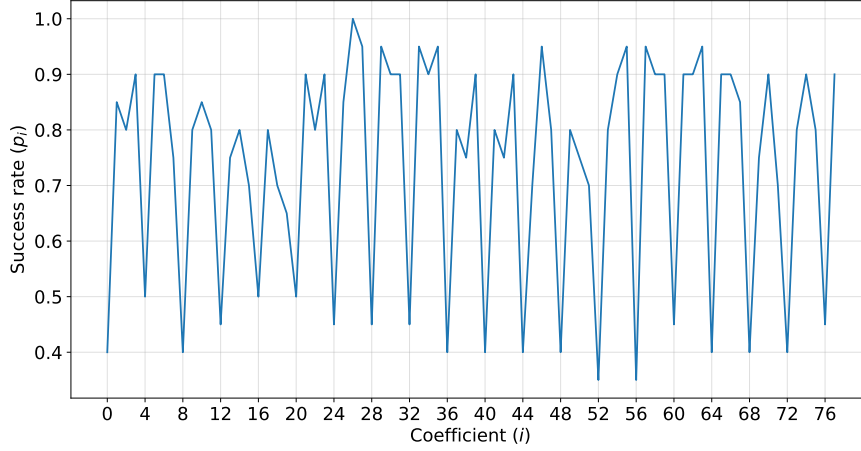
To successfully recover a row i of the secret oil space $\bar{\mathbf{O}}[i :] = \mathbf{A}_i[i :]\mathbf{X}_i^{-1}$, we need that all o faults on index i to be successful, meaning that $\mathbf{A}_i[i :]$ must be correct in every entry. This happens with probability p_i^o .

Our strategy for verifying that all faults for a given index were successful is to generate the row $\bar{\mathbf{O}}[i :]$ multiple times until we see the same row generated twice. In the following, we will explain why this technique is reliable and how we can measure our confidence that the two equal candidates for $\bar{\mathbf{O}}[i :]$ really are produced from two instances where all faults were successful.

Assume we generate l candidate rows $r_1, \dots, r_l$ for $\bar{\mathbf{O}}[i :]$. The probability that the true oil row appears at least twice among these l rows, denoted as event TO_i , can be expressed as:

$$\begin{aligned} \Pr(\text{TO}_i) &= 1 - \Pr(\text{exactly one } r_j \text{ is the true oil}) - \Pr(\text{no } r_j \text{ is the true oil}) \\ &= 1 - \binom{l}{1} p_i^o (1 - p_i^o)^{l-1} - \binom{l}{0} (1 - p_i^o)^l. \end{aligned}$$

However, there is also a chance of a random collision between two incorrect oil rows, which would lead us to incorrectly identify this as the recovered $\bar{\mathbf{O}}[i :]$.

Figure 3: Success rates p_i of best fault parameter F_i for each index i

Let FO_i be the event of finding at least one collision of false oil vectors $r_j = r_k \neq \bar{\mathbf{O}}[i:]$ for $j \neq k$ in l rows. Every time we create a candidate row for $\bar{\mathbf{O}}[i:]$, the vector $\mathbf{A}_i[i:]$ is multiplied with an $\mathbf{X}_i^{-1}$ sampled uniformly at random. The wrong r_j 's are therefore distributed uniformly at random among the $q^o - 1$ wrong candidates. The probability of FO_i is then

$$\begin{aligned} \Pr(\text{FO}_i) &= \sum_{j=2}^l \Pr(\text{A false oil vector appears } j \text{ times among } l \text{ rows}) \\ &= \sum_{j=2}^l \binom{l}{j} \frac{(1 - p_i^o)^j}{(q^o - 1)^{j-1}}. \end{aligned}$$

Let C be the event of observing a row r_j multiple times. We want to find $\Pr(\text{TO}_i \mid \text{C})$, which is the probability that r_j is the correct oil, given that some row has occurred multiple times. By Bayes' rule, we know

$$\Pr(\text{TO}_i \mid \text{C}) = \frac{\Pr(\text{TO}_i \cap \text{C})}{\Pr(\text{C})}.$$

Since $\Pr(\text{TO}_i \cap \text{C}) = \Pr(\text{TO}_i)$ and $\Pr(\text{C}) = \Pr(\text{TO}_i) + \Pr(\text{FO}_i)$ we can substitute this back into the Bayes' theorem formula, to get

$$\Pr(\text{TO}_i \mid \text{C}) = \frac{1 - l \cdot p_i^o (1 - p_i^o)^{l-1} - (1 - p_i^o)^l}{1 - l \cdot p_i^o (1 - p_i^o)^{l-1} - (1 - p_i^o)^l + \sum_{j=2}^l \binom{l}{j} \frac{(1 - p_i^o)^j}{(q^o - 1)^{j-1}}}. \quad (7)$$

We experimentally computed, for each p_i between 0.35 and 1, the number l of r_j 's required to obtain the first repeating row by simulating our attack 1000 times on a signature for which we know the secret key. The average value of l needed before $r_l = r_j$ for some $j < l$ is denoted $\bar{l}_i$. Given $\bar{l}_i$ and an average signing duration on the board of 2.3 seconds (see start of Section 4.2) we can estimate the time $\bar{t}_i = 2.3 \cdot o \cdot \bar{l}_i$ seconds required to collect enough signatures to recover the i -th row. After observing the collisions, we verified whether the obtained row was correct or not. We define $\hat{s}_i$ to be the observed rate where $r_j = r_l$ were the true oil vectors. We also compare this number to the theoretical value

from (7) if we choose l to be $\bar{l}_i$. We present all these values for each p_i between 0.35 and 1 in Table 5. We can see that the theoretical estimate is very close to our actual experimental results, and that we can reliably perform the attack as long as $p_i \geq 0.35$. Looking at Figure 3 we can see that this is the case for all indices.

We can use these values to estimate the number of additional signatures and verifications required in an actual attack for which the secret is unknown, and to determine the overall success probability of our attack. To get the additional factor μ needed to adapt our theoretical attack from Section 3 in practice, sum all the $\bar{l}_i$ together

$$\mu = \sum_{i=0}^{n-o-1} \bar{l}_i.$$

Since we need each index o times, the total number of required signatures is given by

$$o \cdot \mu = 8 \cdot 54097 = 432776.$$

This results in approximately $2^{18.7}$ required signatures and $2^{21.7}$ verifications for **MAYO-1**. Since the verification process can be executed in parallel off-board and at high speeds, the practical duration of the attack is primarily dictated by the time required to collect the faulty signatures. In our experimental setup, faulting one signature takes 2.3 seconds, so collecting enough signatures would require approximately 11.52 days.

Note that the attack efficiency is highly sensitive to the success rate of the fault injection. As shown in Table 5, small improvements, such as employing a more precise faulting technique or a more exhaustive parameter search, could drastically reduce the required time. For instance, increasing the lowest success rates p_{52} and p_{56} from 0.35 to 0.5 would accelerate the attack by a factor of 17 for these indices.

To compute the overall success probability s of the attack, we multiply all the $\hat{s}_i$'s together

$$s = \prod_{i=0}^{n-o-1} \hat{s}_i.$$

The overall success probability of our attack on **MAYO-1** is therefore

$$s = 0.952.$$

Table 5: Experimental results for attack performance and comparison with expected success rate given by Equation (7)

p_i	$\#p_i$ (frequency of p_i)	$\bar{l}_i$ (Avg. l for first collision)	$\bar{l}_i$ (Avg. time to recover row in h)	$\hat{s}_i$ (observed TO rate)	$Pr(TO_i C)$ for $l = \bar{l}_i$
0.35	2	8918	45.6	0.983	0.985
0.4	9	3031	15.5	0.998	0.998
0.45	6	1136	5.8	1	1
0.5	3	514	2.6	1	1
0.55	0	244	1.2	1	1
0.6	0	118	0.6	1	1
0.65	1	61	0.3	1	1
0.7	5	33	0.17	1	1
0.75	6	18	0.092	1	1
0.8	13	12	0.061	1	1
0.85	4	7	0.036	1	1
0.9	20	4	0.02	1	1
0.95	8	3	0.015	1	1
1	1	2	0.01	1	1

5 Countermeasures

In this section, we analyze the effectiveness of various countermeasures against our proposed fault correction attack on randomized MAYO and discuss potential adaptations required to bypass them.

5.1 Comparison of Signature and Vinegar Vectors

In [SMA⁺24], the authors proposed verifying if the generated signatures differ from the initially produced vinegar vectors, thereby ensuring that oil vectors are properly included. While effective against attacks where the addition of the entire oil vector is omitted, this measure is ineffective against our proposed attack, as our fault model involves skipping only a partial addition of the oil vector. Consequently, signatures will consistently differ from vinegar vectors, making this countermeasure insufficient.

Furthermore, approximately $1/q$ of the coefficients in the chosen oil vectors will be zero during normal execution, resulting in some signature coefficients being equal to the corresponding vinegar coefficients. Extending the check described above to individual coefficients will therefore not work, as correctly generated signatures would almost always be flagged as faulty if no vinegar coefficients were allowed in the signature.

5.2 Sign-Then-Verify

The sign-then-verify countermeasure described in [KPLG24] for Dilithium aims to detect signature inconsistencies by performing verification post-generation. In the attack in [KPLG24], it is bypassed using so-called *ineffective faults*. As mentioned above, some coefficients in the chosen oil vectors will be zero during normal execution. Skipping them is called an ineffective fault. In this case, it is revealed that the targeted component was 0, yet it still produced a valid signature. By repeatedly signing and attempting corrections with $\alpha = 0$ until verification succeeds, attackers exploit ineffective faults to circumvent verification-based defenses. However, in our case, this strategy would not circumvent this countermeasure because in order to build the matrix $\mathbf{A}_i$ as in (5) with $a_{i,0}, a_{i,1}, \dots, a_{i,o-1} = 0$ we need o linearly independent solutions $\mathbf{x} \in \mathbb{F}_q^o$ to

$$\bar{\mathbf{O}}[i:] \mathbf{x} = 0, \quad (8)$$

for every $i \in \{0, 1, \dots, n - o\}$. Assuming that such solutions exist, we can build $\mathbf{A}_i$ and $\mathbf{X}_i$ as in Section 3.2. But that also means that

$$\bar{\mathbf{O}}[i:] \mathbf{X}_i = 0,$$

has to hold, which requires $\bar{\mathbf{O}}[i:]$ to be a row consisting only of zeros. This can not be the case for every row, so there is at least one i where we can not find o linearly independent solutions $\mathbf{x} \in \mathbb{F}_q^o$ to (8).

5.3 Shuffling

Current implementations of MAYO sequentially perform the addition of vinegar and oil vector components within a deterministic loop structure as in Listing 1. Introducing randomness into this loop, by shuffling the sequence of additions and potentially including randomization in between the components of each vector as well, obscures the exact position targeted by fault injection.

Nevertheless, attackers may circumvent this countermeasure by exhaustively testing corrections for all potential coefficients of all vectors until the correct one is identified through successful verification. It is important to disregard ineffective faults during this

process, as they always lead to successful verification and hence provide no information about the specific location of the injected fault, which is required for our attack. This can be achieved by excluding the trivial correction $\alpha = 0$ and considering only $\alpha \in \mathbb{F}_q \setminus \{0\}$. This would only slightly increase the number of signatures needed, which can be ignored; however, it would also require a significant amount of additional verification. The total time for the attack then increases to

$$\text{Total Time} \approx N_{\text{sig}} \cdot t_{\text{sign}} + q/2 \cdot k(n - o) \cdot N_{\text{sig}} \cdot t_{\text{verify}}.$$

Note that circumventing this countermeasure for a similar attack on UOV would have relatively lower overhead, because the signature there consists of only a single vector rather than k as in MAYO. The number of verifications needed to circumvent this countermeasure is given in Table 6. While this increases attack complexity, it remains practically feasible if an attacker can run multiple verifications in parallel.

Table 6: Number verifications needed to circumvent shuffling for MAYO and UOV

Algorithm	Verifications
MAYO-1	$\approx 2^{22}$
MAYO-2	$\approx 2^{21}$
MAYO-3	$\approx 2^{23}$
MAYO-5	$\approx 2^{24}$
uov-Ip	$\approx 2^{25}$
uov-Is	$\approx 2^{22}$
uov-III	$\approx 2^{27}$
uov-V	$\approx 2^{28}$

6 Conclusion and future work

We have presented a novel fault-injection attack targeting randomized MAYO signatures and demonstrated its practical feasibility, assuming an attacker can reliably place targeted faults. We demonstrate that the attack can be performed using precise clock glitching to induce faults during signature generation on an STM32F3 microcontroller. By carefully tuning the glitch offset and pulse width parameters, one can introduce faults to skip the secret addition step. These faults allow the extraction of partial secret information, and by exploiting linear dependencies among the oil vectors, one can efficiently extract the secret key.

We also demonstrated that the countermeasure proposed in [SMA⁺24], which checks whether the output signature equals either the oil or the vinegar vector, is ineffective against our attack. Beyond that, our attack is effective against the countermeasure of randomly shuffling indices, albeit at an increased cost. The only effective countermeasure that protects against our correction attack is the sign-and-verify method.

For future work, it would be interesting to evaluate the practical aspects of the attack using different fault-injection methods, such as voltage, electromagnetic, or laser-based techniques, with the goal of increasing the success rate. Testing the attack for higher optimization levels is another interesting direction to investigate. It would also be valuable to study whether the proposed correction attack can be applied to other multivariate signature schemes, such as SNOVA and QR-UOV, and, more generally, to post-quantum signature schemes beyond the multivariate setting.

References

- [ABF⁺03] C. Aumüller, P. Bier, W. Fischer, P. Hofreiter, and J.-P. Seifert. Fault attacks on rsa with crt: Concrete results and practical countermeasures. In Burton S. Kaliski, getin K. Koç, and Christof Paar, editors, *Cryptographic Hardware and Embedded Systems - CHES 2002*, pages 260–275, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.
- [ACK25] Thomas Aulbach, Fabio Campos, and Juliane Krämer. Sok: On the physical security of uov-based signature schemes. In Ruben Niederhagen and Markku-Juhani O. Saarinen, editors, *Post-Quantum Cryptography*, pages 199–231, Cham, 2025. Springer Nature Switzerland.
- [ADN⁺10] Michel Agoyan, Jean-Max Dutertre, David Naccache, Bruno Robisson, and Assia Tria. When clocks fail: On critical paths and clock faults. In Dieter Gollmann, Jean-Louis Lanet, and Julien Iguchi-Cartigny, editors, *Smart Card Research and Advanced Application*, pages 182–193, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.
- [AKKM22] Thomas Aulbach, Tobias Kovats, Juliane Krämer, and Soundes Marzougui. Recovering rainbow’s secret key with a first-order fault attack. In Lejla Batina and Joan Daemen, editors, *Progress in Cryptology - AFRICACRYPT 2022*, pages 348–368, Cham, 2022. Springer Nature Switzerland.
- [AMSU24] Thomas Aulbach, Soundes Marzougui, Jean-Pierre Seifert, and Vincent Quentin Ulitzsch. Mayo or may-not: Exploring implementation security of the post-quantum signature scheme mayo against physical attacks. In *2024 Workshop on Fault Detection and Tolerance in Cryptography (FDTC)*, pages 28–33, 2024.
- [BCD⁺23] Ward Beullens, Ming-Shing Chen, Jintai Ding, Boru Gong, Matthias J. Kannwischer, Jacques Patarin, Bo-Yuan Peng, Dieter Schmidt, Cheng-Jhih Shih, Chengdong Tao, and Bo-Yin Yang. Uov. Technical report, National Institute of Standards and Technology, 2023. Technical report.
- [Beu22] Ward Beullens. Mayo: Practical post-quantum signatures from oil-and-vinegar maps. In Riham AlTawy and Andreas Hülsing, editors, *Selected Areas in Cryptography*, pages 355–376, Cham, 2022. Springer International Publishing.
- [BSK25] Sven Bauer, Fabrizio De Santis, and Kristjane Koleci. Fault attacks against UOV-based signatures. *Cryptology ePrint Archive*, Paper 2025/2101, 2025.
- [DF04] David S. Dummit and Richard M. Foote. *Abstract Algebra, Third Edition*. John Wiley and Sons, 2004.
- [DYC⁺08] Jintai Ding, Bo-Yin Yang, Chia-Hsin Owen Chen, Ming-Shing Chen, and Chen-Mou Cheng. New differential-algebraic attacks and reparametrization of rainbow. In Steven M. Bellovin, Rosario Gennaro, Angelos Keromytis, and Moti Yung, editors, *Applied Cryptography and Network Security*, pages 242–257, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.
- [FKNT22] Hiroki Furue, Yutaro Kiyomura, Tatsuya Nagasawa, and Tsuyoshi Takagi. A new fault attack on uov multivariate signature scheme. In Jung Hee Cheon and Thomas Johansson, editors, *Post-Quantum Cryptography*, pages 124–143, Cham, 2022. Springer International Publishing.

- [GOW⁺24] Ivan Gavrilan, Felix Oberhansl, Alexander Wagner, Emanuele Strieder, and Andreas Zankl. Impeccable keccak: Towards fault resilient sphincs+ implementations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024:154–189, 03 2024.
- [HTS11] Yasufumi Hashimoto, Tsuyoshi Takagi, and Kouichi Sakurai. General fault attacks on multivariate public key cryptosystems. In Bo-Yin Yang, editor, *Post-Quantum Cryptography*, pages 1–18, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- [IMS⁺22] Saad Islam, Koksul Mus, Richa Singh, Patrick Schaumont, and Berk Sunar. Signature correction attack on dilithium signature scheme. In *2022 IEEE 7th European Symposium on Security and Privacy (EuroS&P)*, pages 647–663, 2022.
- [JD25] Sönke Jendral and Elena Dubrova. MAYO key recovery by fixing vinegar seeds. *IACR Communications in Cryptology*, 1(4), 2025.
- [JLK⁺23] Jonas Juffinger, Lukas Lamster, Andreas Kogler, Maria Eichlseder, Moritz Lipp, and Daniel Gruss. Csi:rowhammer – cryptographic security and integrity against rowhammer. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 1702–1718, 2023.
- [KL19] Juliane Krämer and Mirjam Loiero. Fault attacks on uov and rainbow. In *International Workshop on Constructive Side-Channel Analysis and Secure Design*, 2019.
- [KPLG24] Elisabeth Krahmer, Peter Pessl, Georg Land, and Tim Güneysu. Correction fault attacks on randomized crystals-dilithium, Jul. 2024. Artifact available at <https://artifacts.iacr.org/tches/2024/a19>.
- [MBM20] Hassen Mestiri, Imen Barraaj, and Mohsen Machhout. A high-speed keccak architecture resistant to fault attacks. In *2020 32nd International Conference on Microelectronics (ICM)*, pages 1–4, 2020.
- [Nat24a] National Institute of Standards and Technology. Module-lattice-based digital signature standard. Tech. Rep. NIST FIPS 204, National Institute of Standards and Technology, Gaithersburg, MD, August 2024.
- [Nat24b] National Institute of Standards and Technology. Stateless hash-based digital signature standard. Tech. Rep. NIST FIPS 205, National Institute of Standards and Technology, Gaithersburg, MD, August 2024.
- [SK20] Kyung-Ah Shim and Namhun Koo. Algebraic fault analysis of uov and rainbow with the leakage of random vinegar values. *IEEE Transactions on Information Forensics and Security*, 15:2429–2439, 2020.
- [SMA⁺24] Oussama Sayari, Soundes Marzougui, Thomas Aulbach, Juliane Krämer, and Jean-Pierre Seifert. Hamayo: A fault-tolerant reconfigurable hardware implementation of the mayo signature scheme. In Romain Wacquez and Naofumi Homma, editors, *Constructive Side-Channel Analysis and Secure Design*, pages 240–259, Cham, 2024. Springer Nature Switzerland.
- [VR13] van Woudenberg J. Velegalati R., Van Spyk R. Electro magnetic fault injection in practice. In *International Cryptographic Module Conference (ICMC)*, 2013.